Java

Raccolta Completa: 8 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

✨ Fonte Unica per Google NotebookLM

1. Origine e Filosofia di Java

Java nasce nei primi anni ‘90 nei laboratori di **Sun Microsystems** (successivamente acquisita da Oracle) per opera di un team guidato da **James Gosling**.

Inizialmente battezzato con il nome di *Oak* (quercia), il linguaggio venne poi rinominato **Java** nel 1995 durante una sessione di brainstorming davanti a una tazza di caffè indonesiano (da cui derivano il celebre logo della tazza fumante e il termine *“Java Bean”*).

A differenza del C++ (progettato per dare il massimo controllo sull’hardware a discapito della portabilità), Java nasceva con l’obiettivo rivoluzionario di superare la frammentazione dei sistemi operativi: ==W¨rite Once, Run Anywhere=¨= (Scrivi una volta, esegui ovunque)

Oggi Java è ampiamente open-source attraverso il progetto ****OpenJDK****, e non richiede l’acquisto di licenze proprietarie per lo sviluppo standard.

---

2. Il Modello di Esecuzione Ibrido: Compilatore e Bytecode

Spesso ci si chiede: *Java è un linguaggio compilato o interpretato?*
La risposta è: **è un sistema ibrido a due stadi**.

```
flowchart TD
    S["Codice Sorgente Java\n(Programma.java)"] -->|Compilatore javac| B["Bytecode F

    B --> JVM_WIN["JVM per Windows\n(Traduce per x86/x64)"]
    B --> JVM_MAC["JVM per macOS\n(Traduce per Apple Silicon / ARM)"]
    B --> JVM_LIN["JVM per Linux\n(Traduce per Kernel Linux)"]

    JVM_WIN --> WIN["CPU / Hardware Windows"]
    JVM_MAC --> MAC["CPU / Hardware Mac"]
    JVM_LIN --> LIN["CPU / Hardware Linux"]

    style S fill:#2b5b84,stroke:#306998,color:#fff
    style B fill:#ffe873,stroke:#ffd43b,color:#000
    style JVM_WIN fill:#4b8bbe,stroke:#306998,color:#fff
    style JVM_MAC fill:#4b8bbe,stroke:#306998,color:#fff
    style JVM_LIN fill:#4b8bbe,stroke:#306998,color:#fff
```

1. **Fase di Compilazione Statica (`javac`)**:
   Il file sorgente con estensione `.java` viene analizzato dal compilatore `javac` . Il risultato non è un eseguibile binario nativo (come accade in C++ con i file `.exe`), bensì un file binario intermedio e indipendente dall'architettura hardware chiamato `**Bytecode**` (con estensione `.class`).
2. **Fase di Esecuzione Virtuale (JVM)**:
   Il bytecode viene preso in carico dalla `**Java Virtual Machine (JVM)**` installata sul computer dell'utente finale, che lo interpreta ed esegue.

---

3. L'Ecosistema Java: JDK, JRE e JVM a Confronto

Spesso gli studenti confondono queste tre sigle fondamentali. Possiamo immaginarle come tre **scatole cinesi concentriche**:

```
flowchart TD
    subgraph JDK ["JDK (Java Development Kit) - Per i Programmatori"]
        DEV["Compilatore javac, Debugger jdb, Javadoc, Strumenti di Build"]
        subgraph JRE ["JRE (Java Runtime Environment) - Per gli Utenti"]
            LIB["Librerie Standard Java (java.lang, java.util, java.io)"]
            subgraph JVM ["JVM (Java Virtual Machine)"]
                ENG["Class Loader + Bytecode Interpreter + JIT Compiler + Garbage Co
            end
        end
    end

    style JDK fill:#e1f5fe,stroke:#03a9f4,color:#01579b
    style JRE fill:#fff9c4,stroke:#fbc02d,color:#f57f17
    style JVM fill:#d4edda,stroke:#28a745,color:#155724
```

Componente	Nome Esteso	Cosa Contiene?	A Chi Serve?
JVM	*Java Virtual Machine*	Motore software che interpreta il bytecode, compila le parti critiche a runtime (JIT) e gestisce la RAM col Garbage Collector.	È il componente che **esegue** effettivamente le istruzioni.
JRE	*Java Runtime Environment*	La **JVM** + tutte le **librerie standard di runtime** precompilate (Class Library).	Serve all'utente finale per poter **eseguire i programmi Java** sul proprio PC.
JDK	*Java Development Kit*	Il **JRE** + tutti gli **strumenti per lo sviluppatore**: compilatore `javac`, archiviatore `jar`, debugger `jdb`, `javadoc`.	Serve al programmatore per **scrivere e compilare** nuovo codice Java.

---

4. Tabella Comparativa: Java vs C++ vs Python

Caratteristica	C++	Java	Python
Paradigma	Multi-paradigma a basso/alto livello	**OOP puro** (tutto risiede in una classe)	Multi-paradigma ad altissimo livello
Compilazione	Compilato nativo diretto ad oggetti macchina (`.exe`)	**Ibrido** : Bytecode `.class` su **JVM** con JIT	Bytecode eseguito su interprete PVM
Portabilità	Bassa (va ricompilato per ogni OS)	**Massima (WORA)** : stesso `.class` su ogni OS	Alta (richiede interprete Python installato)
Tipizzazione	Statica e forte	**Statica e forte** (controlli ferrei a compile-time)	Dinamica e forte
Gestione Memoria	Manuale (`new` / `delete` , puntatori espliciti)	**Automatica con Garbage Collector**	Automatica con Reference Counting e GC
Puntatori	Puntatori espliciti con aritmetica (`*` , `&`)	**Riferimenti gestiti** (nessuna aritmetica puntatori)	Riferimenti trasparenti a oggetti

5. Il Primo Programma: “Hello World” in Java

In Java **qualsiasi riga di codice deve obbligatoriamente risiedere all'interno di una classe** . Il nome del file deve corrispondere esattamente al nome della classe pubblica principale!

> 💉 **Programma Completo:** `HelloWorld.java`

```java
// Il file DEVE chiamarsi esattamente "HelloWorld.java"
public class HelloWorld {

    // Punto di ingresso dell'applicazione (Main entry point)
    public static void main(String[] args) {
        System.out.println("========================================
        System.out.println("   Benvenuto nel mondo della programmazio
        System.out.println("========================================

        // Stampa dettagli dell'ambiente di esecuzione JVM
        String versioneJava = System.getProperty("java.version");
        String sistemaOperativo = System.getProperty("os.name");

        System.out.println("Versione Java in uso : " + versioneJava);
        System.out.println("Sistema Operativo    : " + sistemaOperati
        System.out.println("Compilazione Bytecode: Riuscita con succe
    }
}
```

Istruzioni da Terminale:

1. Compilazione: `javac HelloWorld.java` (genera il file `HelloWorld.class`).
2. Esecuzione: `java HelloWorld` (avvia la JVM che esegue il bytecode).

1. Anatomia del Metodo `main`

In Java ogni programma eseguibile parte da una firma rigorosa e standardizzata:

```
public class Calcolatrice {
    // Firma universale del metodo principale
    public static void main(String[] args) {
        // Istruzioni eseguite dalla JVM
    }
}
```

- **`public`**: il metodo è accessibile dall'esterno da parte della JVM.
- **`static`**: il metodo appartiene alla classe e può essere invocato dalla JVM senza dover prima istanziare un oggetto.
- **`void`**: non restituisce alcun valore al sistema operativo.
- **`main`**: il nome convenzionale del punto di avvio.
- **`String[] args`**: array di stringhe contenente gli eventuali argomenti passati da riga di comando.

---

2. Gli 8 Tipi di Dato Primitivi

Mentre in C e C++ la dimensione di un `int` o `long` può variare a seconda del compilatore e del processore (32 o 64 bit), **in Java le dimensioni sono scolpite nello standard del linguaggio** per garantire la portabilità assoluta.

```
flowchart TD
    TIPI["Tipi di Dato in Java"]
    TIPI --> PRIM["1. Tipi Primitivi (Allocati nello Stack)"]
    TIPI --> REF["2. Tipi Riferimento (Allocati nello Heap)"]

    PRIM --> NUM_INT["Interi: byte, short, int, long"]
    PRIM --> NUM_DEC["Decimali: float, double"]
    PRIM --> TESTO["Carattere: char (Unicode UTF-16)"]
    PRIM --> LOGIC["Logico: boolean (true / false)"]

    REF --> STR["String (Testo immutabile)"]
    REF --> ARR["Array (int[], String[]...)"]
    REF --> CLS["Istanze di Classi e Oggetti"]

    style PRIM fill:#cce5ff,stroke:#007bff,color:#004085
    style REF fill:#ffe873,stroke:#ffd43b,color:#000
```

Tipo	Dimensione	Note & Regole di Sintassi
`byte`	8 bit (1 byte)	Utile per buffer grezzi di byte e socket di rete.
`short`	16 bit (2 byte)	
`int`	32 bit (4 byte)	Tipo predefinito per numeri interi.
`long`	64 bit (8 byte)	**Richiede obbligatoriamente il suffisso `L`**.
`float`	32 bit (4 byte)	**Richiede obbligatoriamente il suffisso `f`**.
`double`	64 bit (8 byte)	Tipo predefinito standard per numeri decimali.
`char`	16 bit (2 byte)	Apici singoli `'...'`. **Supporta caratteri internazionali ed emoji** !
`boolean`	1 bit logico	**NON è un numero intero** (nessun cast da/verso 0 e 1 come in C++!).

---

3. Tipi Riferimento e Classi Wrapper

In Java i tipi primitivi non sono oggetti (per massimizzare la velocità nei calcoli elementari nello Stack). Tuttavia, le strutture dati dinamiche della libreria standard (come `ArrayList`) richiedono necessariamente oggetti.

A tale scopo, ogni tipo primitivo possiede una corrispondente ****Classe Wrapper**** (*Involucro*):

- `int` → **`Integer`**
- `double` → **`Double`**
- `char` → **`Character`**
- `boolean` → **`Boolean`**

Meccanismo di Autoboxing e Unboxing

Java esegue la conversione automatica e trasparente tra tipo primitivo e classe wrapper:

```
// Autoboxing: da primitivo int a oggetto Integer nello Heap
Integer objNumero = 42;

// Unboxing: da oggetto Integer a primitivo int nello Stack
int primitivo = objNumero;
```

---

4. La Classe `String` e la Trappola del Confronto

In Java le stringhe non sono semplici array di caratteri terminati da `\0` come in C, ma sono istanze della classe **`java.lang.String`**.

🔒 Immutabilità delle Stringhe

Le stringhe in Java sono totalmente immutabili. Qualsiasi metodo (come `.toUpperCase()`) non modifica la stringa originale, ma **genera e restituisce un nuovo oggetto nello Heap**:

```
String saluto = "Ciao";
saluto.toUpperCase();
System.out.println(saluto); // Stampa ancora "Ciao" (l'originale non

saluto = saluto.toUpperCase(); // Riassegnazione: ora punta al nuovo
```

🚫 Errore Gravissimo: Confrontare Stringhe con `==`

L'operatore `==` confronta se due variabili puntano allo **stesso indirizzo fisico di memoria nello Heap**, NON il contenuto testuale!
Per verificare se due stringhe contengono le stesse lettere, usa **SEMPRE** **`.equals()`**:

```
String s1 = "Java";
String s2 = new String("Java");

System.out.println(s1 == s2);      // FALSE! (Puntano a due oggetti d
System.out.println(s1.equals(s2));  // TRUE!  (Stesso identico testo
```

---

5. Conversioni di Tipo (Casting) e Parsing

A. Casting Implicito ed Esplicito

- **Widening (Allargamento implicito)**: assegnazione da tipo più piccolo a uno più capiente senza perdita di precisione (es. da `int` a `double`):

```
int x = 10;
double d = x; // Automatico: d vale 10.0
```

- **Narrowing (Restringimento esplicito)**: forzatura manuale con cast tra parentesi tonde, rischiando troncamento:

```
double pi = 3.99;
int intero = (int) pi; // Tronca la parte decimale: intero vale 3!
```

B. Parsing da Stringa a Numero

Quando si leggono dati da input o da file, arrivano sotto forma di testo. Per convertirli in valori numerici usiamo i metodi statici dei wrapper:

```
String testoEtà = "18";
String testoPrezzo = "19.99";

int eta = Integer.parseInt(testoEtà);        // Converte in int primitivo
double prezzo = Double.parseDouble(testoPrezzo); // Converte in double pri
```

---

6. Input e Output da Console

A. Output con `System.out`

- `System.out.println(dato)` : stampa il valore e aggiunge automaticamente un a-capo.
- `System.out.print(dato)` : stampa mantenendo il cursore sulla stessa riga.
- `System.out.printf("Formato %d %.2f\n", intero, decimale)` : stampa formattata.

B. Input Interattivo con `java.util.Scanner`

Per leggere da tastiera usiamo la classe `Scanner` collegata a `System.in` :

```
import java.util.Scanner;

public class TestInput {
    public static void main(String[] args) {
        Scanner tastiera = new Scanner(System.in);

        System.out.print("Inserisci il tuo nome: ");
        String nome = tastiera.nextLine(); // Legge l'intera riga spazi in

        System.out.print("Inserisci l'età: ");
        int eta = tastiera.nextInt(); // Legge un intero

        System.out.println("Benvenuto " + nome + ", hai " + eta + " anni."
        tastiera.close();
    }
}
```

⚠️ La Trappola del Buffer di `Scanner`

Se chiami `tastiera.nextLine()` subito dopo `tastiera.nextInt()` o `tastiera.nextDouble()`, `nextLine()` leggerà immediatamente una **stringa vuota** saltando l'input dell'utente!
Causa: `nextInt()` legge il numero ma lascia il carattere Invio (`\n`) in sospeso nel buffer della tastiera.
Soluzione obbligatoria: inserisci sempre un **`tastiera.nextLine();` a vuoto** per ripulire il buffer prima di leggere testo:

```
int matricola = tastiera.nextInt();
tastiera.nextLine(); // ==CONSUMA IL BUFFER RESIDUO \n==
String nome = tastiera.nextLine(); // Ora legge correttamente la stri
```

---

7. Programma Esecutivo Completo

🧪 Programma Completo: Calcolatore di Media Ponderata Studente

```java
import java.util.Scanner;

public class GestoreMedia {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.println("=== CALCOLATORE MEDIA SCOLASTICA ===");

        System.out.print("Nome studente: ");
        String nome = input.nextLine();

        System.out.print("Materia principale: ");
        String materia = input.nextLine();

        System.out.print("Voto scritto (1-10): ");
        double votoScritto = input.nextDouble();

        System.out.print("Voto orale (1-10): ");
        double votoOrale = input.nextDouble();

        // Calcolo media ponderata: scritto 40%, orale 60%
        double mediaPonderata = (votoScritto * 0.40) + (votoOrale * 0
        boolean promosso = mediaPonderata >= 6.0;

        System.out.println("\n-----------------------------------");
        System.out.printf("Studente        : %s%n", nome.toUpperCase(
        System.out.printf("Disciplina      : %s%n", materia);
        System.out.printf("Media Calcolata : %.2f%n", mediaPonderata)
        System.out.printf("Esito Finale    : %s%n", (promosso ? "PROM
        System.out.println("-----------------------------------");

        input.close();
    }
}
```

1. Selezione Condizionale: `if`, `else if`, `else`

La sintassi delle decisioni in Java è basata sulle parentesi graffe `{}` e sulle condizioni booleane rigorose:

```
int punteggio = 85;

if (punteggio >= 90) {
    System.out.println("Grado: Eccellente");
} else if (punteggio >= 60) {
    System.out.println("Grado: Sufficiente");
} else {
    System.out.println("Grado: Insufficiente");
}
```

> ⚠️ **Condizioni Rigorosamente Booleane**
>
> In C++ un intero 0 equivale a falso e qualsiasi numero diverso da zero a vero (es. `if (x)`). In Java questo genera un errore di compilazione: l'espressione dentro `if(...)` **deve produrre obbligatoriamente un tipo primitivo boolean** (`if (x != 0)`).

Operatore Ternario

Assegnazione rapida monoriga: variabile = (condizione) ? valoreSeVero : valoreSeFalso;

```
int eta = 19;
String stato = (eta >= 18) ? "Maggiorenne" : "Minorenne";
```

---

2. Selezione Multipla: `switch` Tradizionale vs Moderno

A. Lo Switch Classico (con `break`)

```
int mese = 2;
switch (mese) {
    case 1:
        System.out.println("Gennaio");
        break; // ==OBBLIGATORIO: senza break si scivola nel caso sotto (fa
    case 2:
        System.out.println("Febbraio");
        break;
    default:
        System.out.println("Altro mese");
}
```

B. Enhanced Switch con Freccia (Java 14+)

Nelle versioni moderne di Java si usa la **notazione a freccia `->`**: elimina il rischio di dimenticare il `break`, permette di raggruppare più casi con la virgola e può restituire direttamente un valore:

```
int giorno = 6;

String tipoGiorno = switch (giorno) {
    case 1, 2, 3, 4, 5 -> "Giorno Lavorativo";
    case 6, 7          -> "Fine Settimana";
    default            -> "Giorno non valido";
};

System.out.println(tipoGiorno); // Fine Settimana
```

---

3. I Cicli Iterativi e il `for-each`

A. Cicli Classici: `while`, `do-while` e `for`

- `while (cond)`: verifica prima di eseguire (0 o più volte).
- `do { ... } while (cond);`: esegue almeno una volta, poi verifica la condizione a valle.
- `for (int i = 0; i < n; i++)`: ciclo standard con contatore deterministico.

B. Il Ciclo Avanzato `for-each`

È la modalità consigliata e idiomatica per scorrere tutti gli elementi di un array o di una collezione, evitando errori di indici fuori limite:

```
int[] numeri = {10, 20, 30, 40, 50};

// Sintassi: for (TipoVariabile elemento : contenitore)
for (int num : numeri) {
    System.out.print(num + " ");
}
// Output: 10 20 30 40 50
```

> **NOTE**
>
> Il ciclo **`for-each` è di sola lettura**: non permette di modificare i valori originali dell'array né di conoscere l'indice di posizione numerico dell'elemento corrente.

---

4. Gli Array in Java: Oggetti nello Heap

A differenza del C++ dove un array `int v[10]` riserva semplicemente 40 byte contigui nello Stack, **in Java qualsiasi array è un oggetto a tutti gli effetti allocato nello Heap**.

---

1. Dichiarazione e Allocazione Dinamica

```
// 1. Dichiarazione (nello Stack viene creata solo la variabile di riferim
int[] temperature;

// 2. Allocazione nello Heap tramite la keyword new
temperature = new int[7]; // ==Vengono allocate 7 celle, inizializzate aut

// La proprietà .length ci dà la dimensione esatta
System.out.println("Giorni registrati: " + temperature.length); // 7
```

2. Inizializzazione Immediata

Possiamo dichiarare e popolare un array contemporaneamente:

```
String[] studenti = {"Alessandro", "Beatrice", "Carlo", "Diana"};
```

3. Matrici (Array Multidimensionali)

Una matrice in Java è letteralmente un *“array di array”*:

```
// Griglia 3 righe x 3 colonne
int[][] matrice = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

System.out.println(matrice[1][2]); // Riga 1, Colonna 2 -> Stampa 6
```

---

8. Programma Esecutivo Completo

> 🧪 **Programma Completo: Analisi Statistica Voti con Array e Ordinamento**

```java
import java.util.Scanner;
import java.util.Arrays;

public class StatisticheClasse {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Quanti studenti compongono la classe? ");
        int n = scanner.nextInt();

        double[] voti = new double[n];

        // Caricamento dei voti da tastiera
        for (int i = 0; i < voti.length; i++) {
            System.out.printf("Inserisci voto studente %d: ", (i + 1)
            voti[i] = scanner.nextDouble();
        }

        // Calcolo di somma e media con ciclo for-each
        double somma = 0;
        for (double v : voti) {
            somma += v;
        }
        double media = somma / voti.length;

        // Ordinamento dell'array tramite Arrays.sort
        Arrays.sort(voti);
        double votoMinimo = voti[0];
        double votoMassimo = voti[voti.length - 1];

        // Stampa dei risultati
        System.out.println("\n=======================================
        System.out.println("       REPORT STATISTICO CLASSE        ")
        System.out.println("======================================="
        System.out.println("Tutti i voti ordinati: " + Arrays.toStrin
        System.out.printf("Numero totale studenti: %d%n", voti.length
        System.out.printf("Media complessiva     : %.2f%n", media);
        System.out.printf("Voto più basso        : %.2f%n", votoMinim
        System.out.printf("Voto più alto         : %.2f%n", votoMassi
        System.out.println("======================================="
```

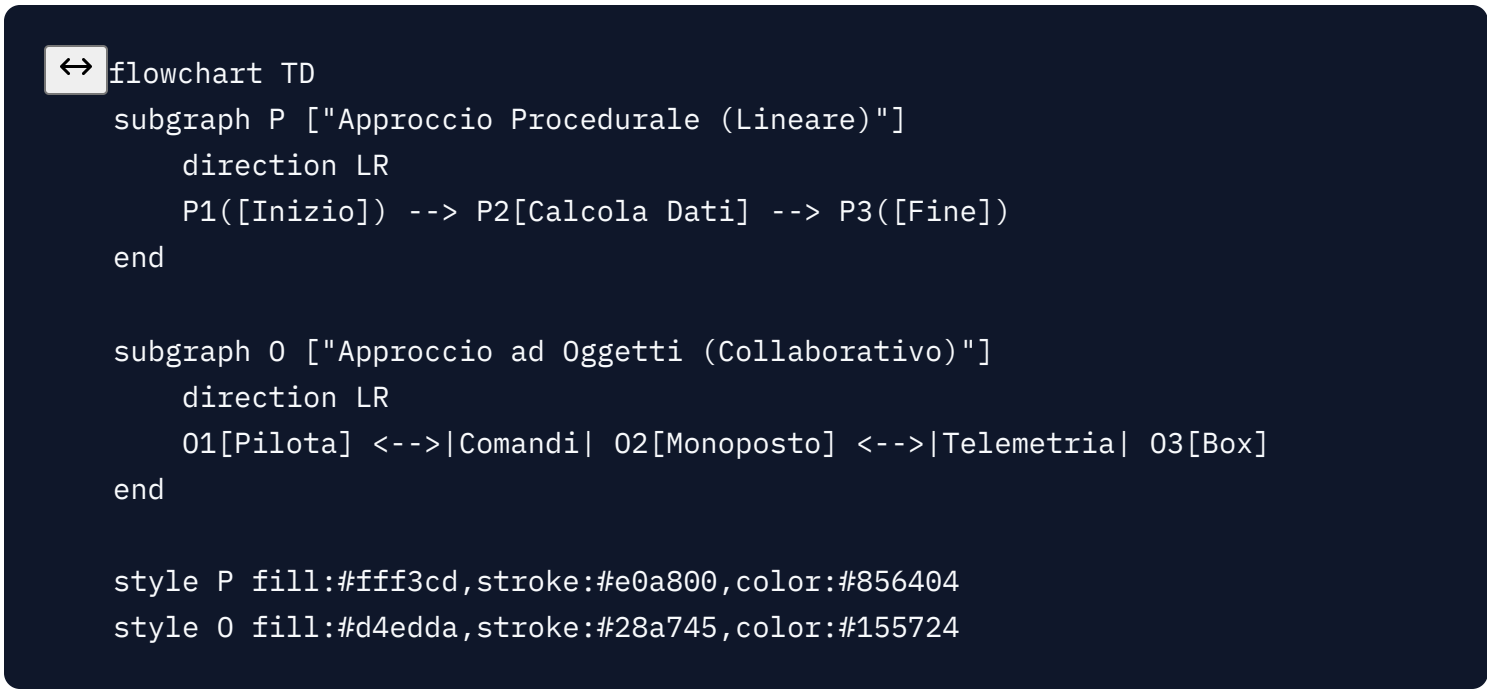
```
        scanner.close();
    }
}
```

1. La Rivoluzione della OOP: Dai Flowchart alle Classi

Nella programmazione procedurale classica (come nel linguaggio C), l'attenzione era focalizzata sull'**algoritmo sequenziale**: passi logici, condizioni e cicli rappresentati graficamente tramite *flowchart*.

Nella **Programmazione Orientata agli Oggetti (OOP)**, la progettazione parte invece dalla risposta a due domande:

1. *Quali entità (oggetti) compongono il mio dominio applicativo?*
2. *Quali caratteristiche possiedono e come interagiscono tra loro?*



- `**Classe**`: è il **modello astratto**, il progetto architettonico o stampo (es. la scheda tecnica di una vettura di Formula 1).
- `**Oggetto (o Istanza)**`: è l'**esemplare reale, tangibile**, generato a partire dalla classe e residente nella memoria Heap (es. la singola monoposto con telaio #16 guidata da Leclerc).

---

2. Anatomia di una Classe in UML (Software NClass)

Nei progetti software professionali le classi si progettano visivamente con lo standard universale `**UML (Unified Modeling Language)**`, utilizzando strumenti visuali come **NClass**.

Nel diagramma UML, una classe è rappresentata graficamente come una scatola divisa in **tre scomparti orizzontali**:

---

1. **Scomparto Superiore (Intestazione)**:
   Contiene il **Nome della Classe**, scritto obbligatoriamente con la prima lettera maiuscola in notazione **PascalCase** (es. `Monoposto`, `ContoCorrente`).
2. **Scomparto Centrale (Attributi / Campi)**:
   Elenca le variabili di istanza che definiscono lo **stato** dell'oggetto.
   Sintassi UML: `visibilità nomeAttributo : Tipo` (es. `- cavalli : int`).
3. **Scomparto Inferiore (Metodi / Operazioni)**:
   Elenca le funzioni che definiscono il **comportamento** dell'oggetto.
   Sintassi UML: `visibilità nomeMetodo(parametro : Tipo) : TipoRitorno` (es. `+ accelera(delta : int) : int`).

---

3. I 4 Modificatori di Accesso (Visibilità)

In Java e nei diagrammi UML, ogni attributo e metodo è preceduto da un simbolo che stabilisce chi ha il permesso di leggerlo o invocarlo:

Simbolo UML	Modificatore Java	Livello di Visibilità	Spiegazione Dettagliata
`-`	**`private`**	**Solo la Classe stessa**	I membri privati sono invisibili a chiunque all'esterno della classe (pilastro dell'Incapsulamento).
`+`	**`public`**	**Tutto il Mondo**	Accessibili liberamente da qualsiasi altra classe di qualsiasi package dell'applicazione.
`#`	**`protected`**	**Package + Sottoclassi**	Accessibili dalle classi nello stesso package e da tutte le classi figlie che ereditano, ovunque siano.
	(Default)	**Solo lo Stesso Package**	Detto **Package-Private**. Se non scrivi alcun modificatore, il membro è visibile solo alle classi situate nella stessa identica cartella/package.

---

4. Incapsulamento: Metodi Query (Getter) e Modificatori (Setter)

L'**Incapsulamento (*Information Hiding*)** è la regola cardine della OOP:
==Gli attributi devono essere SEMPRE privati (-) e mai accessibili direttamente dall'esterno!==

Se un attributo fosse `public`, qualsiasi codice esterno potrebbe impostare dati privi di senso, corrompendo lo stato dell'oggetto (es. `auto.velocita = -300;` oppure `auto.cavalli = 0;`).

Per interagire in sicurezza con gli attributi privati, la classe fornisce metodi pubblici specializzati:

```
flowchart LR
    CLIENT["Codice Esterno (Main)"] -->|1. Richiesta lettura| GET["Metodo Query (Get
    CLIENT -->|2. Richiesta modifica| SET["Metodo Modificatore (Setter)\nsetVelocita
    GET -->|Legge| ATTR["Attributo Privato\n- velocita"]
    SET -->|Scrive se valido| ATTR
    style ATTR fill:#f8d7da,stroke:#dc3545,color:#721c24
    style GET fill:#cce5ff,stroke:#007bff,color:#004085
    style SET fill:#d4edda,stroke:#28a745,color:#155724
```

1. **Metodi Query (o Getter / Ispettori)**:
   Hanno lo scopo di **interrogare e leggere** lo stato dell'oggetto restituendo il valore dell'attributo, senza modificarlo:

```
public int getVelocita() {
    return this.velocita;
}
```

2. **Metodi Modificatori (o Setter)**:
   Hanno lo scopo di **aggiornare** il valore dell'attributo, **applicando tassativamente controlli di coerenza e validazione** prima della scrittura:

```
public void setVelocita(int nuovaVelocita) {
    if (nuovaVelocita >= 0 && nuovaVelocita <= 380) {
        this.velocita = nuovaVelocita;
    } else {
        System.out.println("⚠️ Valore velocità non consentito!");
    }
}
```

---

5. Codice Java Completo e Testabile

>

🧪 Programma Completo: `Monoposto.java` e `SimulatoreGara.java`

```
// Definizione della Classe Monoposto
public class Monoposto {
    // 1. Attributi privati (Information Hiding)
    private int cavalli;
    private float peso;
    private String colore;
    private int velocita;

    // 2. Costruttore
    public Monoposto(int cavalli, float peso, String colore) {
        this.cavalli = cavalli;
        this.peso = peso;
        this.colore = colore;
        this.velocita = 0; // Parte da ferma
    }

    // 3. Metodi operativi (Comportamenti)
    public int accelera(int delta) {
        if (delta > 0) {
            this.velocita += delta;
            System.out.println("🏎️ La monoposto accelera a " + this.v
        }
        return this.velocita;
    }

    public void frena() {
        this.velocita = Math.max(0, this.velocita - 50);
        System.out.println("🛑 Frenata! Nuova velocità: " + this.velo
    }

    // 4. Metodi Query (Getter) e Modificatori (Setter)
    public int getVelocita() {
        return this.velocita;
    }

    public void setVelocita(int velocita) {
        if (velocita >= 0 && velocita <= 370) {
            this.velocita = velocita;
        } else {
            System.out.println("❌ Errore: velocità fuori limite rego
        }
```

```
    }

    public String getColore() {
        return this.colore;
    }
}
```

```
// Classe Main di esecuzione
public class SimulatoreGara {
    public static void main(String[] args) {
        // Istanziazione dell'oggetto nello Heap
        Monoposto f1 = new Monoposto(1000, 798.0f, "Rosso Corsa");

        System.out.println("Colore vettura: " + f1.getColore());
        System.out.println("Velocità di partenza: " + f1.getVelocita(

        // Simulazione azioni
        f1.accelera(120);
        f1.accelera(160);
        f1.frena();

        // Test validazione incapsulamento
        f1.setVelocita(-40); // Rifiutato dal setter!
    }
}
```

1. Cos'è un Costruttore

Il **Costruttore** è il componente deputato a far nascere un oggetto in uno stato coerente e garantito.

In Java, il costruttore ha due caratteristiche sintattiche inconfondibili:

1. Ha **esattamente lo stesso nome della classe** (rispettando maiuscole e minuscole).
2. **Non specifica alcun tipo di ritorno**, nemmeno la parola chiave `void`.

```
public class Smartphone {
    private String marca;
    private int batteria;

    // Costruttore della classe Smartphone
    public Smartphone(String marca, int batteria) {
        this.marca = marca;
        this.batteria = batteria;
    }
}
```

---

2. Le Tre Tipologie di Costruttori

```
flowchart TD
    COST["Costruttori in Java"]
    COST --> DEF["1. Costruttore di Default\nSmartphone()\nSenza parametri: valori p
    COST --> PAR["2. Costruttore Parametrico\nSmartphone(marca, batt)\nPersonalizzaz
    COST --> COP["3. Costruttore di Copia\nSmartphone(altroPhone)\nClonazione profor

    style COST fill:#2b5b84,stroke:#306998,color:#fff
    style DEF fill:#cce5ff,stroke:#007bff,color:#004085
    style PAR fill:#d4edda,stroke:#28a745,color:#155724
    style COP fill:#fff3cd,stroke:#ffc107,color:#856404
```

1. Costruttore di Default (senza parametri)

Inizializza gli attributi con valori neutri o predefiniti di fabbrica.

```
public Monoposto() {
    this.cavalli = 800;
    this.peso = 798.0f;
    this.colore = "Nero Test";
    this.velocita = 0;
}
```

⚠️ La Regola del Costruttore di Default Automatico

Java ti regala un costruttore vuoto invisibile **solo finché non ne scrivi nessuno**.
Non appena scrivi un qualsiasi costruttore parametrico, il costruttore vuoto gratuito scompare all'istante!

```
// ❌ CODICE CHE GENERA ERRORE (Il costruttore vuoto è scomparso)
class Monoposto {
    int cavalli;
    // Abbiamo scritto SOLO il costruttore parametrico:
    public Monoposto(int cavalli) {
        this.cavalli = cavalli;
    }
}

// Nel Main:
Monoposto m1 = new Monoposto(900); // ✅ OK (usa il costruttore param
Monoposto m2 = new Monoposto();    // ❌ ERRORE DI COMPILAZIONE!
                                   // "The constructor Monoposto() is
```

✅ Come si fa a supportare ENTRAMBI?

Per poter creare sia oggetti predefiniti (`new Monoposto()`) sia oggetti personalizzati (`new Monoposto(1000, "Rosso")`), devi semplicemente **scrivere ENTRAMBI i costruttori dentro la stessa classe**:

```
class Monoposto {
    int cavalli;
    String colore;

    // 1. Costruttore di DEFAULT (scritto esplicitamente da noi!)
    public Monoposto() {
        this.cavalli = 800;
        this.colore = "Nero Standard";
    }

    // 2. Costruttore PARAMETRICO (per valori personalizzati)
    public Monoposto(int cavalli, String colore) {
        this.cavalli = cavalli;
        this.colore = colore;
    }
}

// Nel Main ora funzionano ENTRAMBI in totale armonia:
Monoposto base  = new Monoposto();              // ✅ Chiama il Cost
Monoposto corsa = new Monoposto(1000, "Rosso");  // ✅ Chiama il Cost
```

2. Costruttore Parametrico

Permette a chi istanzia l'oggetto di **specificare i valori desiderati per i campi** al momento della nascita:

```
public Monoposto(int cavalli, float peso, String colore) {
    this.cavalli = cavalli;
    this.peso = peso;
    this.colore = colore;
    this.velocita = 0;
}
```

3. Costruttore di Copia (*Copy Constructor*)

Se scriviamo `Monoposto b = a;` , **non stiamo duplicando la vettura**: abbiamo solo creato due puntatori nello Stack che guardano la stessa identica macchina nello Heap (**effetto aliasing**). Modificando `b` , cambierà anche `a` !

Per creare una ****vera copia indipendente**** usiamo il costruttore di copia:

```
public Monoposto(Monoposto sorgente) {
    // Clona tutti i valori dell'oggetto sorgente nel nuovo oggetto
    this.cavalli = sorgente.cavalli;
    this.peso = sorgente.peso;
    this.colore = sorgente.colore;
    this.velocita = sorgente.velocita;
}
```

---

3. La Parola Chiave `this`

La parola chiave ****`this`**** è un riferimento implicito all'**istanza corrente** su cui è stato invocato il codice. Ha due utilizzi capitali:

A. Disambiguazione di Nomi (Shadowing)

Quando il nome del parametro formale del costruttore è identico al nome dell'attributo della classe, l'attributo viene "nascosto" (*shadowed*). Per indicare al compilatore l'attributo usiamo **`this.`** :

```
public void setPeso(float peso) {
    this.peso = peso; // this.peso = attributo istanza; peso = argomento de
}
```

B. Chiamata a Cascata tra Costruttori (`this(...)`)

Per evitare di duplicare codice nei vari costruttori sovraccaricati, un costruttore può chiamare un altro costruttore della medesima classe:

```
public class Monoposto {
    private int cavalli;
    private float peso;
    private String colore;

    // Costruttore parametrico completo
    public Monoposto(int cavalli, float peso, String colore) {
        this.cavalli = cavalli;
        this.peso = peso;
        this.colore = colore;
    }

    // Costruttore di default che riutilizza quello sopra (DEVE essere la
    public Monoposto() {
        this(850, 798.0f, "Bianco Standard"); // ==Invocazione del costrut
    }
}
```

---

4. Ciclo di Vita in Memoria RAM: Cosa fa la keyword `new`?

Quando scriviamo `Monoposto f1 = new Monoposto(1000, 798f, "Rosso");`, la JVM esegue 4 passaggi sequenziali:

```
flowchart LR
    subgraph STACK ["Memoria Stack (Variabili Locali)"]
        REF["f1\n(Riferimento: 0x8A10)"]
    end

    subgraph HEAP ["Memoria Heap (Oggetti Dinamici)"]
        OBJ["Oggetto Monoposto (0x8A10)\n- cavalli = 1000\n- peso = 798.0\n- colore
    end

    REF -->|Punta a| OBJ

    style REF fill:#4b8bbe,stroke:#306998,color:#fff
    style OBJ fill:#ffe873,stroke:#ffd43b,color:#000
```

1. **Allocazione**: riserva un blocco contiguo di byte nello Heap sufficiente a contenere tutti gli attributi della classe.
2. **Inizializzazione di Default**: azzera tutti i bit dei campi (numeri a 0, booleani a `false`, riferimenti a `null`).
3. **Esecuzione Costruttore**: esegue le istruzioni del corpo del costruttore passato con i parametri specificati.
4. **Assegnazione Riferimento**: l'indirizzo esadecimale dell'oggetto allocato nello Heap viene memorizzato nella variabile `f1` nello Stack.

---

5. Placeholder per NClass e Istruzioni Pratiche

> 🖼️ **Placeholder Immagine: Costruttori e Overload su NClass**
>
> - **File da inserire nel vault**: `![[java_uml_costruttori_nclass.png|550]]`
> - **Cosa ricercare su Google / Web**:
>   *"NClass multiple constructors overloading diagram"*, *"UML constructor notation class diagram"*.
> - **Come disegnarlo passo-passo su NClass**:
>   1. Nella classe creata su NClass, fai clic destro → **Add Constructor**.
>   2. Crea il primo costruttore (Default): nessun parametro, Modifier: `Public (+)`.
>   3. Fai nuovamente clic destro → **Add Constructor** per il secondo (Parametrico):

- Arguments: `int cavalli, float peso, string colore`.

4. Aggiungi il terzo costruttore (Di Copia):
   - Arguments: `Monoposto sorgente`.
5. NClass mostrerà i tre costruttori raggruppati all'inizio della sezione metodi, evidenziando il meccanismo di **Overload dei costruttori**.

---

6. Domande d'Esame e Concetti Chiave

❓ Domande di Verifica

1. *Cosa accade se si antepone `void` alla firma di un costruttore (es. `public void Monoposto()`)?*
   Risposta: Il compilatore non lo considera più un costruttore, bensì un **comune metodo della classe**. Di conseguenza, all'atto del `new Monoposto()` quel metodo non verrà invocato e Java cercherà un costruttore reale, sollevando un errore se assente.
2. *Perché l'istruzione `Monoposto b = a;` non crea una vera copia dell'oggetto?*
   Risposta: Perché le variabili di tipo classe in Java contengono solo riferimenti (indirizzi di memoria). Assegnare una variabile all'altra copia solo l'indirizzo, facendo sì che entrambe puntino allo **stesso unico oggetto nello Heap**.
3. *Dove deve trovarsi la chiamata `this(...)` all'interno di un costruttore sovraccaricato?*
   Risposta: Deve trovarsi tassativamente come **prima riga di codice** del corpo del costruttore; se collocata in posizione successiva, il compilatore rifiuta il codice.

---

7. Codice Java Completo e Testabile

💉 Programma Completo: Dimostrazione dei 3 Costruttori e Verifica Anti-Aliasing

```
public class TestCostruttori {
    public static void main(String[] args) {
        System.out.println("=== 1. TEST COSTRUTTORE PARAMETRICO ===")
        Monoposto auto1 = new Monoposto(950, 795.0f, "Verde Aston");
        auto1.accelera(150);
        System.out.println("Auto 1: " + auto1.getColore() + " | Vel:

        System.out.println("\n=== 2. TEST COSTRUTTORE DI DEFAULT ==="
        Monoposto autoDefault = new Monoposto();
        System.out.println("Auto Default: " + autoDefault.getColore()

        System.out.println("\n=== 3. TRAPPOLA DELL'ALIASING VS COSTRU
        // Aliasing errato (copia solo il riferimento)
        Monoposto aliasAuto1 = auto1;
        aliasAuto1.setVelocita(280);
        System.out.println("Auto 1 originale dopo modifica di alias:

        // Vera clonazione con Costruttore di Copia
        Monoposto copiaReale = new Monoposto(auto1);
        copiaReale.setVelocita(320);
        System.out.println("Auto 1 originale dopo copia reale     : "
        System.out.println("Copia reale indipendente              : "
    }
}
```

1. La Mappa delle Relazioni in UML

Nel linguaggio visivo UML, le relazioni tra classi definiscono l'accoppiamento e la gerarchia del sistema:

```
classDiagram
    direction LR
    class Scuderia {
        - nome : String
        + ingaggiaPilota(p : Pilota)
    }
    class Pilota {
        - nome : String
        - numero : int
    }
    class Monoposto {
        - telaio : String
    }
    class Motore {
        - cilindrata : int
        - cavalli : int
    }
    class MeccanicoTelemetria {
        + analizza(m : Monoposto) void
    }

    Scuderia "1" o-- "*" Pilota : Aggregazione (◇ Rombo Vuoto)\nIl pilota esiste an
    Scuderia "1" *-- "2" Monoposto : Composizione (◆ Rombo Pieno)\nLe monoposto app
    Monoposto "1" *-- "1" Motore : Composizione (◆ Rombo Pieno)\nIl motore nasce e
    MeccanicoTelemetria ..> Monoposto : Dipendenza (--> Tratteggiata)\nUsa la vettu
```

---

2. Analisi Dettagliata delle Relazioni

1. Dipendenza (*Dependency* - relazione `\"uses-a\"`)

È la relazione più debole e temporanea. La classe A **utilizza temporaneamente** la classe B all'interno di un suo metodo, ma non la conserva come attributo permanente.

- **Tipologia d'uso**: B viene passata come **parametro formale di un metodo**, oppure viene istanziata come variabile locale provvisoria.
- **Simbolo UML**: **freccia tratteggiata aperta (`- - - →`)**.

```
// Il meccanico "dipende" da Monoposto solo durante la diagnosi
public class Meccanico {
    public void effettuaTagliando(Monoposto m) { // Ricevuta come parametr
        System.out.println("Revisione della monoposto completata.");
    }
}
```

---

2. Associazione (*Association* - relazione \"knows-a\")

È una relazione strutturale stabile. La classe A memorizza un riferimento permanente a un oggetto di classe B all'interno dei suoi attributi.

- **Molteplicità (Cardinalità)**: indica quanti oggetti partecipano al legame:
  - `1`: esattamente uno.
  - `0..1`: zero oppure uno (opzionale).
  - `*` o `0..*`: zero, uno o molti (gestiti tramite array o `ArrayList`).
  - `1..*`: da uno a molti.

```
public class Pilota {
    private String nome;
    private Monoposto vetturaAssegnata; // Associazione: il pilota "conosce
}
```

---

3. Aggregazione (*Aggregation* - relazione \"has-a\" debole)

L'Aggregazione è una forma specializzata di associazione che esprime un rapporto **tutto-parti**: il contenitore possiede uno o più elementi, ma **il loro ciclo di vita è indipendente**.

- **Regola di Sopravvivenza**: se l'oggetto contenitore viene deallocato o distrutto, gli oggetti contenuti **NON vengono distrutti**, ma continuano a esistere autonomamente nella memoria Heap.
- **Simbolo UML**: **rombo vuoto (bianco)** `◇` posizionato dal lato del contenitore.

```
public class Scuderia {
    private String nome;
    private Pilota primoPilota; // Aggregazione

    // Il pilota viene creato FUORI e passato alla scuderia
    public Scuderia(String nome, Pilota p) {
        this.nome = nome;
        this.primoPilota = p;
    }
    // Se la Scuderia fallisce e viene cancellata,
    // l'oggetto Pilota continua a esistere e può passare ad un'altra squa
}
```

---

4. Composizione (*Composition* - relazione \"has-a\" forte)

La Composizione è la forma più forte e vincolante di associazione tutto-parti.

- **Ciclo di Vita Coincidente**: l'oggetto componente nasce con il contenitore e **se il contenitore muore, il componente viene distrutto all'istante**. Non ha senso che il componente esista da solo.
- **Simbolo UML**: **rombo pieno (nero)** `◆` posizionato dal lato del contenitore.

```
public class Monoposto {
    private String modello;
    private Motore motoreInterno; // Composizione: ciclo di vita legato

    public Monoposto(String modello, int cilindrata) {
        this.modello = modello;
        // L'oggetto Motore viene istanziato DIRETTAMENTE all'interno della
        this.motoreInterno = new Motore(cilindrata);
    }
    // Se la Monoposto viene deallocata, il Motore viene eliminato dal Garb
}
```

---

5. Generalizzazione (*Generalization* - relazione \"is-a\")

Rappresenta l'**ereditarietà** tra una superclasse generale e una sottoclasse specifica.

- La sottoclasse “è una” versione specializzata della classe base.
- **Simbolo UML**: linea continua con **freccia a triangolo vuoto** `—▷` che punta verso la classe genitore.
- *Esempio*: la classe `Monoposto` è una generalizzazione di `VeicoloDaCorsa`.

---

3. Codice Java Completo e Testabile

> 🧪 **Programma Completo: Modello di Simulazione Formula 1 con tutte le Relazioni**

```
// 1. Componente esclusivo per la Composizione
class Motore {
    private int cavalli;
    public Motore(int cavalli) { this.cavalli = cavalli; }
    public int getCavalli() { return cavalli; }
}

// 2. Entità indipendente per l'Aggregazione
class Pilota {
    private String nome;
    private int numero;
    public Pilota(String nome, int numero) {
        this.nome = nome;
        this.numero = numero;
    }
    public String getNome() { return nome; }
}

// 3. Classe Monoposto: COMPOSIZIONE con Motore
class Monoposto {
    private String modello;
    private Motore motore; // Composizione: possesso forte

    public Monoposto(String modello, int cavalli) {
        this.modello = modello;
        this.motore = new Motore(cavalli); // Nasce con la monoposto
    }
    public String getInfo() {
        return modello + " (Motore V6 Turbo " + motore.getCavalli() +
    }
}

// 4. Classe Scuderia: AGGREGAZIONE con Pilota
class Scuderia {
    private String nome;
    private Pilota pilotaUfficiale; // Aggregazione: possesso debole

    public Scuderia(String nome, Pilota p) {
        this.nome = nome;
        this.pilotaUfficiale = p; // Il pilota viene agganciato dall'
    }
```

```
    public void mostraSchieramento() {
        System.out.println("Scuderia " + nome + " schiera il pilota:
    }
}

// 5. Classe Ingegnere: DIPENDENZA con Monoposto
class IngegnerePista {
    // Dipendenza: riceve la monoposto solo come parametro provvisori
    public void controllaTelemetria(Monoposto m) {
        System.out.println("📊 Telemetria analizzata per: " + m.getIn
    }
}

public class MainRelazioni {
    public static void main(String[] args) {
        // Creiamo il pilota autonomamente
        Pilota charles = new Pilota("Charles Leclerc", 16);

        // Aggregazione: la scuderia aggrega il pilota
        Scuderia ferrari = new Scuderia("Scuderia Ferrari", charles);
        ferrari.mostraSchieramento();

        // Composizione: la vettura crea internamente il suo motore
        Monoposto sf24 = new Monoposto("Ferrari SF-24", 1020);

        // Dipendenza: l'ingegnere esegue il check-up
        IngegnerePista ing = new IngegnerePista();
        ing.controllaTelemetria(sf24);
    }
}
```

1. Il Meccanismo dell'Ereditarietà (`extends`)

L' `**Ereditarietà**` è il principio della OOP che modella la relazione gerarchica `**"is-a"** ("è un")` .
Consente a una nuova classe (**sottoclasse** o classe figlia) di acquisire attributi e metodi da una classe preesistente (**superclasse** o classe genitore), specializzandone il comportamento ed evitando duplicazioni di codice.

```
classDiagram
    class Veicolo {
        <<abstract>>
        # marca : String
        # modello : String
        # anno : int
        + getDettagli() String
        + calcolaBollo()* double
    }
    class Auto {
        - numeroPorte : int
        + calcolaBollo() double
    }
    class Moto {
        - tempiMotore : int
        + calcolaBollo() double
    }

    Veicolo <|-- Auto : extends (—▷ Triangolo Vuoto)
    Veicolo <|-- Moto : extends (—▷ Triangolo Vuoto)
```

In Java:

```
// Classe genitore (Superclasse)
public class Veicolo {
    protected String marca;
    protected String modello;
}

// Classe figlia (Sottoclasse) che eredita tramite extends
public class Auto extends Veicolo {
    private int numeroPorte; // Attributo specifico di Auto
}
```

> ⚠️ **Divieto di Ereditarietà Multipla di Classi**
>
> In C++ una classe può ereditare contemporaneamente da due o tre classi, creando complessi problemi di ambiguità (il celebre *Diamond Problem*).
> **Java** **vieta l'ereditarietà multipla tra classi** : ogni classe può avere una sola superclasse diretta (`extends` accetta un solo nome). Per ereditare comportamenti multipli, Java adotta le **Interfacce**.

---

2. Il Costruttore Figlio e la Chiamata a `super()`

Quando nasce un oggetto di una sottoclasse, la porzione ereditata della classe genitore deve essere inizializzata per prima. A questo scopo si usa **`super(...)`** :

```
public class Veicolo {
    protected String marca;
    protected String modello;

    public Veicolo(String marca, String modello) {
        this.marca = marca;
        this.modello = modello;
    }
}

public class Auto extends Veicolo {
    private int porte;

    public Auto(String marca, String modello, int porte) {
        super(marca, modello); // ==Invoca il costruttore di Veicolo (TASS
        this.porte = porte;
    }
}
```

---

3. Il Modificatore di Accesso `protected`

Se un attributo della superclasse è dichiarato `private`, la sottoclasse non potrà accedervi direttamente (potrà farlo solo tramite getter pubblici).

Dichiarando l'attributo **`protected`**, permettiamo alle classi figlie di accedervi direttamente senza esporlo al mondo esterno:

```
public class Veicolo {
    protected int velocitaMassima; // Accessibile da Veicolo e da tutte le
}
```

---

4. Classi e Metodi Astratti (`abstract`)

In una gerarchia di veicoli ha senso istanziare un'automobile o una motocicletta, ma che senso avrebbe istanziare un "Veicolo" generico? Nessuno, perché è un'idea astratta!

A. Classe Astratta (`abstract class`)

È una classe contrassegnata dalla parola chiave **`abstract`**.

- Definisce caratteristiche e logiche comuni a tutta la gerarchia.
- **NON può essere istanziata direttamente**: l'istruzione **`new Veicolo()` genererà un errore di compilazione** !
- Può contenere sia metodi normali con codice, sia metodi astratti.

B. Metodo Astratto (`abstract method`)

È un metodo che possiede solo la **firma**, ma **non ha alcun corpo** (non ha le graffe `{}` e termina con `;`):

```
public abstract double calcolaBollo(); // Obbliga i figli a implementarlo!
```

Ogni sottoclasse concreta che estende la classe astratta ha l' **obbligo contrattuale di implementare il metodo astratto** (facendone l'overriding), altrimenti non potrà essere compilata.

---

5. La Parola Chiave `final`

La keyword **`final`** rappresenta l'opposto di `abstract` :

1. **Variabile** `final` : diventa una **costante non riassegnabile** (`final double PI = 3.14159;`).
2. **Metodo** `final` : **impedisce alle sottoclassi di sovrascrivere** quel metodo (blocca l'overriding).
3. **Classe** `final` : classe terminale che **non può essere estesa** da nessun'altra sottoclasse (ad esempio, la classe standard `String` è `final` per motivi di integrità e sicurezza).

---

6. Placeholder per NClass e Istruzioni Pratiche

🖼️ Placeholder Immagine: Gerarchia e Notazione Astratta su NClass

- **File da inserire nel vault**: `![[java_uml_nclass_classi_astratte.png|550]]`
- **Cosa ricercare su Google / Web**:
  "NClass abstract class method italic representation", *"UML generalization inheritance hierarchy diagram NClass"*.
- **Come disegnarlo passo-passo su NClass**:
  1. Su **NClass**, crea una classe e nominala `Veicolo`.
  2. Seleziona la classe e nella finestra laterale **Properties** imposta: **`IsAbstract = True`**.
     Nota visiva: NClass scriverà immediatamente il titolo della classe in **`**CORSIVO (*Italic*)**`**!
  3. Aggiungi il metodo `calcolaBollo`: nelle sue proprietà imposta **`IsAbstract = True`**. Il nome del metodo apparirà in corsivo.
  4. Crea due classi concrete normali: `Auto` e `Moto`.
  5. Seleziona lo strumento **Generalization** (freccia con triangolo vuoto `–▷`) e trascina la freccia da `Auto` a `Veicolo` e da `Moto` a `Veicolo`.
  6. Esporta l'immagine nel vault con il nome `java_uml_nclass_classi_astratte.png`.

---

7. Domande d'Esame e Concetti Chiave

❓ Domande di Verifica

1. *Cosa accade se una sottoclasse concreta non implementa uno dei metodi astratti ereditati dalla superclasse?*
   Risposta: Il compilatore rifiuta il codice generando un errore, a meno che la sottoclasse stessa non venga a sua volta dichiarata **`abstract`**.
2. *Come viene rappresentata una classe astratta in un diagramma UML disegnato con NClass?*

Risposta: Il nome della classe viene renderizzato graficamente con font in stile `**corsivo (*Italic*)**` e/o con il tag descrittivo `<<abstract>>`.

3. *È possibile che una classe astratta contenga un costruttore se non può essere istanziata direttamente?*
   Risposta: Sì, le classi astratte possono e devono avere costruttori per inizializzare i propri campi protetti, i quali verranno invocati dalle sottoclassi figlie tramite l'istruzione `` `super(...)` ``.

---

8. Codice Java Completo e Testabile

🧪 Programma Completo: Gerarchia Veicoli con Calcolo Tasse

```
// Superclasse Astratta
abstract class Veicolo {
    protected String targa;
    protected int cavalliFiscali;

    public Veicolo(String targa, int cavalliFiscali) {
        this.targa = targa;
        this.cavalliFiscali = cavalliFiscali;
    }

    // Metodo concreto ereditato da tutti
    public void mostraDatiBase() {
        System.out.println("Targa: " + targa + " | Potenza: " + caval
    }

    // METODO ASTRATTO (Ciascun tipo di veicolo calcola la tassa con
    public abstract double calcolaBollo();
}

// Sottoclasse Concreta Auto
class Auto extends Veicolo {
    private boolean elettrica;

    public Auto(String targa, int cavalliFiscali, boolean elettrica)
        super(targa, cavalliFiscali);
        this.elettrica = elettrica;
    }

    @Override
    public double calcolaBollo() {
        if (elettrica) {
            return 0.0; // Esente da bollo
        }
        return cavalliFiscali * 2.58;
    }
}

// Sottoclasse Concreta Moto
class Moto extends Veicolo {
    public Moto(String targa, int cavalliFiscali) {
        super(targa, cavalliFiscali);
```

```
    }

    @Override
    public double calcolaBollo() {
        return 25.0 + (cavalliFiscali * 1.20);
    }
}

public class MainEreditarietà {
    public static void main(String[] args) {
        // Veicolo v = new Veicolo("AA000AA", 100); <-- ERRORE! Classe

        Auto auto1 = new Auto("AB123CD", 110, false);
        Auto autoTesla = new Auto("EV999ZZ", 300, true);
        Moto moto1 = new Moto("MI45678", 75);

        auto1.mostraDatiBase();
        System.out.printf("Bollo Auto Termica: €%.2f%n", auto1.calcola

        autoTesla.mostraDatiBase();
        System.out.printf("Bollo Auto Elettrica: €%.2f%n", autoTesla.

        moto1.mostraDatiBase();
        System.out.printf("Bollo Moto: €%.2f%n", moto1.calcolaBollo()
    }
}
```

1. Cos’è il Polimorfismo

Il termine **Polimorfismo** (dal greco *poly* = molti, *morphé* = forma) è il **terzo pilastro fondamentale** della programmazione ad oggetti, dopo Incapsulamento ed Ereditarietà.

Indica la **capacità di un'entità software di assumere forme e comportamenti differenti** a seconda dell’oggetto a cui si riferisce a tempo di esecuzione.

Si divide in due macro-categorie:

1. **Polimorfismo per i Dati** (Riferimento Polimorfico e Collezioni Eterogenee).
2. **Polimorfismo per i Metodi** (Overloading a compile-time vs Overriding a runtime).

---

2. Polimorfismo per i Dati: Il Riferimento Polimorfico

Grazie all’ereditarietà, una **variabile di riferimento di tipo della classe genitore può puntare a qualsiasi oggetto istanziato da una sua sottoclasse**:

```
// Il tipo statico è Figura, ma l'oggetto dinamico nello Heap è un Cerchio
Figura f1 = new Cerchio(5.0);
Figura f2 = new Rettangolo(4.0, 6.0);
```

```
flowchart LR
    subgraph STACK ["Riferimento di Tipo Base"]
        V["Figura f"]
    end

    subgraph HEAP ["Oggetti Concreti nello Heap"]
        C["Oggetto Cerchio\nraggio = 5.0\ncalcolaArea() -> 78.5"]
        R["Oggetto Rettangolo\nbase = 4, alt = 6\ncalcolaArea() -> 24.0"]
    end

    V -.->|A runtime può puntare a| C
    V -.->|Oppure può puntare a| R

    style V fill:#4b8bbe,stroke:#306998,color:#fff
    style C fill:#d4edda,stroke:#28a745,color:#155724
    style R fill:#fff3cd,stroke:#ffc107,color:#856404
```

La Potenza delle Collezioni Eterogenee

Possiamo **memorizzare elementi di classi diverse in un unico array o collezione** e farli interagire senza dover sapere a priori di che oggetto specifico si tratti:

```
Figura[] forme = {
    new Cerchio(3.0),
    new Rettangolo(2.0, 5.0),
    new Cerchio(10.0)
};

// Polimorfismo in azione: ciascun oggetto calcolerà la propria area con la
for (Figura fig : forme) {
    System.out.println("Area calcolata: " + fig.calcolaArea());
}
```

Operatore `instanceof` e Downcasting

Per verificare a runtime se un riferimento punta a una specifica sottoclasse ed effettuare un **downcasting sicuro**:

```
if (f1 instanceof Cerchio) {
    Cerchio c = (Cerchio) f1; // Downcasting esplicito sicuro
    System.out.println("Raggio del cerchio: " + c.getRaggio());
}
```

---

3. Polimorfismo per i Metodi: Overloading vs Overriding

Caratteristica	Overloading (Sovraccarico)	Overriding (Sovrascrittura)
Dove si applica?	Nella **stessa classe**	Tra **Superclasse e Sottoclasse**
Nome del metodo	Identico	Identico
Lista Parametri	**DIVERSA** (per numero o tipo di dati)	**IDENTICA** per numero e tipi
Tipo di Ritorno	Può essere diverso	Deve essere identico (o covariante)
Momento di Risoluzione	**Compile-Time** (Static / Early Binding)	**Runtime** (Dynamic / Late Binding)
Annotazione	Nessuna	**`@Override`**

Late Binding (Dynamic Dispatch)

Quando invochi `fig.calcolaArea()`, il compilatore non sa quale formula verrà eseguita.
A runtime, la JVM consulta la **tabella virtuale dei metodi (**VMT - Virtual Method Table**)** dell'oggetto effettivo presente nello Heap e invoca la versione sovrascritta corretta.

---

4. Le Interfacce (`interface`)

Se una classe astratta modella *"cosa una classe È"*, un'**Interfaccia** modella **`*"cosa una classe SA FARE"*`** (un'abilità o un **`contratto comportamentale puro`**).

Caratteristiche fondamentali:

- Si dichiara con la parola chiave **`` `interface` ``**.
- Tutti i metodi sono implicitamente **`` `public abstract` ``** (firme pure senza corpo).
- Non possiede variabili di istanza, solo costanti **`` `public static final` ``**.
- **`**Ereditarietà Multipla Permessa**`** : una classe può implementare **`un numero illimitato di interfacce`** tramite la parola chiave **`` `implements` ``**!

```
classDiagram
    class Disegnabile {
        <<interface>>
        + disegna() void
    }
    class Stampabile3D {
        <<interface>>
        + esportaSTL() void
    }
    class Figura {
        <<abstract>>
        + calcolaArea()* double
    }
    class Cerchio {
        - raggio : double
        + calcolaArea() double
        + disegna() void
        + esportaSTL() void
    }

    Figura <|-- Cerchio : extends (Ereditarietà Singola)
    Disegnabile <|.. Cerchio : implements (Tratteggiata - - ▷)
    Stampabile3D <|.. Cerchio : implements (Tratteggiata - - ▷)
```

```
public interface Disegnabile {
    void disegna(); // Metodo astratto
}

public interface Stampabile3D {
    void esportaSTL();
}

// Una sola superclasse estesa, ma MOLTEPLICI interfacce implementate!
public class Cerchio extends Figura implements Disegnabile, Stampabile3D {
    private double raggio;

    public Cerchio(double raggio) { this.raggio = raggio; }

    @Override
    public double calcolaArea() {
        return Math.PI * raggio * raggio;
    }

    @Override
    public void disegna() {
        System.out.println("Disegno un cerchio di raggio " + raggio);
    }

    @Override
    public void esportaSTL() {
        System.out.println("Esportazione mesh 3D del cilindro completata.")
    }
}
```

5. Tabella Comparativa: Classe Astratta vs Interfaccia

Criterio	Classe Astratta (`abstract class`)	Interfaccia (`interface`)
Relazione concettuale	Relazione di appartenenza: **"is-a"** (È un)	Relazione di abilità: **"can-do"** (Sa fare)
Ereditarietà	**Singola**: solo 1 superclasse (`extends`)	**Multipla**: N interfacce (`implements`)
Attributi	Può avere attributi di istanza (`private` , `protected`)	**Solo costanti pubbliche statiche** (`public static final`)
Costruttori	**Sì**, per inizializzare i campi ereditati	**No**, non può avere alcun costruttore
Scopo Primario	Condivisione di stato e codice comune	Garanzia di un protocollo o contratto di comunicazione

---

6. Placeholder per NClass e Istruzioni Pratiche

> 🖼️ **Placeholder Immagine: Interfacce e Realizzazione su NClass**
>
> - **File da inserire nel vault**: `![[java_uml_nclass_interfacce_polimorfismo.png|600]]`
> - **Cosa ricercare su Google / Web**:
>   *"NClass UML interface realization dashed triangle arrow"*, *"Polymorphism UML class diagram interface implementation"*.
> - **Come disegnarlo passo-passo su NClass**:
>   1. Su **NClass**, seleziona lo strumento **Interface** nella barra superiore e posizionalo sul foglio.
>   2. Nominalo `Disegnabile` e aggiungi il metodo `disegna(): void` . NClass apporrà automaticamente lo stereotipo `<>`.
>   3. Crea la classe concreta `Cerchio` .

4. Seleziona lo strumento **Realization** (noto anche come *Implementation*: freccia tratteggiata con triangolo vuoto `- - - ▷`).
5. Trascina la freccia da `Cerchio` fino a `Disegnabile`.
6. Esporta il diagramma e inseriscilo nel vault come `java_uml_nclass_interfacce_polimorfismo.png`.

---

7. Domande d'Esame e Concetti Chiave

❓ Domande di Verifica

1. *Qual è la differenza fondamentale tra Overloading e Overriding in Java?*
   Risposta: L'**Overloading** si realizza nella **stessa classe** e consiste in metodi con lo stesso nome ma **parametri diversi** risolti a **compile-time**. L'**Overriding** consiste nella riscrittura di un metodo della superclasse da parte della sottoclasse con la **stessa identica firma**, risolta a **runtime tramite Late Binding**.
2. *Perché è considerata ottima prassi inserire l'annotazione `@Override` prima di un metodo sovrascritto?*
   Risposta: Perché istruisce il **compilatore a verificare che il metodo stia effettivamente sovrascrivendo** una funzione della superclasse. Se si commette un errore di battitura, il compilatore genera un **errore immediato anziché creare silenziosamente un overload involontario**.
3. *Perché Java non permette l'ereditarietà multipla tra classi ma la permette per le interfacce?*
   Risposta: L'ereditarietà multipla di classi genera ambiguità nel caso due genitori abbiano attributi o implementazioni con lo stesso nome (**Diamond Problem**). Nelle interfacce non ci sono attributi di stato né conflitti di implementazione, quindi una classe può garantire **più contratti comportamentali in assoluta sicurezza**.

---

8. Codice Java Completo e Testabile

🧪 Programma Completo: Sistema Grafico Polimorfico con Interfacce

```
// 1. Interfaccia Comportamentale
interface Disegnabile {
    void disegna();
}

// 2. Superclasse Astratta
abstract class FiguraGeometria {
    protected String colore;
    public FiguraGeometria(String colore) { this.colore = colore; }
    public abstract double calcolaArea();
}

// 3. Sottoclasse Concreta Rettangolo
class Rettangolo extends FiguraGeometria implements Disegnabile {
    private double base;
    private double altezza;

    public Rettangolo(String colore, double base, double altezza) {
        super(colore);
        this.base = base;
        this.altezza = altezza;
    }

    @Override
    public double calcolaArea() { return base * altezza; }

    @Override
    public void disegna() {
        System.out.printf("🟦 Disegno Rettangolo %s [%.1fx%.1f]%n", c
    }
}

// 4. Sottoclasse Concreta Cerchio
class Cerchio extends FiguraGeometria implements Disegnabile {
    private double raggio;

    public Cerchio(String colore, double raggio) {
        super(colore);
        this.raggio = raggio;
    }
```

```
    @Override
    public double calcolaArea() { return Math.PI * raggio * raggio; }

    @Override
    public void disegna() {
        System.out.printf("🔴 Disegno Cerchio %s [Raggio=%.1f]%n", co
    }
}

public class MainPolimorfismo {
    public static void main(String[] args) {
        // Array Polimorfico di tipo Interfaccia Disegnabile
        Disegnabile[] scena = {
            new Rettangolo("Blu", 10.0, 5.0),
            new Cerchio("Rosso", 4.0),
            new Rettangolo("Giallo", 3.0, 3.0)
        };

        System.out.println("=== RENDERING GRAFICO POLIMORFICO ===");
        for (Disegnabile elemento : scena) {
            // Chiamata polimorfica a runtime (Late Binding!)
            elemento.disegna();

            // Se è anche una FiguraGeometria, ne calcoliamo l'area
            if (elemento instanceof FiguraGeometria) {
                FiguraGeometria fg = (FiguraGeometria) elemento;
                System.out.printf("   -> Area calcolata: %.2f mq%n", 
            }
        }
    }
}
```